Your home app is a Python module, hence the __init__.py file. The other skeleton files for the app got created as well. Unfortunately, creating the app is not sufficient. You also need to tell Django about it. The settings.py file in your RiffMates/RiffMates configuration directory is a Python script containing variable declarations that define Django's behavior.

The settings.py file comes with dozens of configuration values, and you can add your own and access them through the `django.config.settings` module. To register your app, modify the `INSTALLED_APPS` configuration value, which is a `list` of strings that are the names of all the apps in your project.

The developers of Django have consciously made a choice here: instead of auto-discovering all the modules and treating them like apps, you have to explicitly register an app. Although this is an extra step, it means Django doesn't have to differentiate between regular Python modules and Django apps, which are also modules. By doing it this way, your project can contain modules that aren't apps, if you wish.

Django has its own apps, which come with the framework. A new project has several of them already included in `INSTALLED_APPS`. To register your newly created home app, open RiffMates/RiffMates/settings.py and add your app's name to the `list`, as in the following listing.

Listing 2.1 Tell Django about your home app

```
# RiffMates/RiffMates/settings.py
...
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'home',
]
...
```

Throughout the book, you'll see … to indicate this listing has code before or after it. Conveniently, this is also valid Python, and your compiler will ignore it if you copy it into your source.

Add the home app to the INSTALLED_APPS list.

Django is now aware of your app. You're ready to add a view to show your Credits page.

2.5 Your first Django view

Each view is responsible for creating content that Django returns when the view's corresponding URL gets visited. Django provides some built-in views, but most of the time, you're going to want your own. You can write views as either functions or classes. I prefer function-based views, as I find it clearer to understand a URL calling a function than it corresponding to one or more methods in a class. I'll be sticking with function-based views throughout the book.